

Project Specification: MICHWAR Ride-Hailing Platform

Role: Act as a Senior Software Architect and Lead Flutter Developer.

Objective: Design and architect a professional, high-performance ride-hailing application called "MICHWAR" for the Algerian market. The app must function as a single, unified codebase that supports both Passenger and Driver modes.

Technical Constraints:

- **Frontend:** Flutter (Dart).
- **Backend:** Firebase (Authentication, Firestore, Cloud Functions).
- **Geospatial:** Use Geohash for proximity-based driver matching (Do not use inefficient standard Firestore queries).
- **Maps:** Google Maps SDK (Maps, Directions, Distance Matrix API).
- **Deployment:** Must be performant, fast, and optimized for high-concurrency.

Core Feature Requirements:

1. **Unified Role-Based Architecture:** A single app that switches UI/UX logic based on user status (Driver or Passenger).
2. **Advanced Driver Tools:**
 - **Heatmaps:** Real-time visual demand overlay.
 - **"Heading Home" Toggle:** Restricts ride requests to those in the driver's target direction.
 - **Adaptive GPS:** Dynamically adjust tracking frequency to save battery.
3. **Safety & Reliability:**
 - **Live Trip Sharing:** Generate temporary, secure web links for real-time tracking.
 - **In-app SOS:** Instant alerts to admin with coordinates.
 - **Graceful Disconnection:** Robust logic to sync ride state immediately after signal restoration.
4. **Financial & Compliance Logic:**
 - **Compliance:** Secure document upload portal for Algerian requirements (License, Carte Grise, etc.).
 - **Tiered Commission:**
 - *Base:* 80% to Driver, 20% to Company.
 - *Elite (100+ rides & 4+ stars):* 90% to Driver, 10% to Company.
 - **Company Surcharge:** An additional 1–4 DZD surcharge added to every ride (100% company revenue).

- **Driver Wallet:** Pre-paid wallet system where commission + surcharge is deducted from the driver's balance. Restricted access if the balance is too low.
 - **Server-Side Validation:** All financial calculations (fare, commission, surcharge, status changes) MUST occur in Firebase Cloud Functions to prevent client-side fraud.
5. **Gamification:** A "MICHWAR Points" system for passengers to unlock discounts or premium tiers.

Your Deliverable:

1. **System Architecture:** Provide a high-level technical map showing how the Flutter frontend interacts with Firebase and the Geohash logic.
2. **Data Schema:** Define the JSON structure for the `users`, `drivers`, `rides`, `transactions`, and `wallet` collections.
3. **Cloud Function Logic:** Provide the pseudo-code/logic for the "Ride Completion" trigger that calculates the final fare, splits the commission, adds the surcharge, updates the driver's wallet, and checks for tier promotion.
4. **App Structure:** Provide a recommended `lib/` directory structure for a feature-first Flutter project.

Please be concise, focus on production-ready patterns, and ensure the logic accounts for the specific financial requirements provided.

1. Project Overview

MICHWAR is a unified mobile application built with **Flutter** that serves both passengers and drivers. The platform aims to provide a professional, fast, and secure ride-hailing experience tailored for the Algerian market.

2. Technical Requirements

- **Frontend:** Flutter (Single app codebase with role-based UI toggling: Driver/Passenger).
- **Backend & Database:** Firebase (Authentication, Firestore, Cloud Functions).
- **Real-time Optimization:** Use **Geohash** (or Redis Geo) for proximity-based queries to ensure the system remains fast under load.
- **Maps & Navigation:** Google Maps SDK (Maps, Directions, Distance Matrix API).
- **Deployment:** Must be optimized for high-concurrency performance and resilient to network instability.

3. Mandatory Feature Set

A. Advanced User Experience

- **Gamified Loyalty Program:** Implement a "MICHWAR Points" system. Users earn points per ride, redeemable for discounts or upgrades (e.g., "Eco-friendly" or "Premium" ride tiers).
- **Predictive Destinations:** Utilize device sensors and historical time-of-day data to suggest likely destinations (e.g., home/work suggestions).

- **Earnings Dashboard (Drivers):** A clean, engaging visual interface showing daily, weekly, and monthly earnings, including breakdowns of tips and bonuses.

B. Safety & Reliability

- **Live Trip Sharing:** Ability to generate a secure, temporary web link for friends/family to track the ride in real-time.
- **In-app SOS:** A prominent emergency button that instantly alerts the admin team and provides live GPS coordinates.
- **Graceful Disconnection:** Robust background logic that caches state during network loss (e.g., tunnels) and syncs instantly upon reconnection.

C. Smart Driver Tools

- **Demand Heatmaps:** Real-time visual overlays showing high-demand areas to help drivers maximize income.
- **"Heading Home" Toggle:** A filter that restricts ride requests to those roughly along the driver's route home.
- **Adaptive GPS Tracking:** Optimize battery usage by dynamically adjusting location update frequency based on ride status (active vs. idle).

4. Algeria-Specific Compliance & Security

To operate effectively within the Algerian regulatory environment, the system must include the following:

- **Driver Verification Workflow:** The app must include a secure document upload module for drivers. Before activation, drivers are required to submit:
 - **National Identity Card**
 - **Driver's License**
 - **Vehicle Registration (Carte Grise)**
 - **Valid Insurance**
 - **Control Technique Certificate**
- **Data Integrity: Server-Side Validation:** All fare calculations, status updates (start/end trip), and loyalty point additions must be validated on the server. Never trust the client (mobile app) to calculate pricing or ride states to prevent fraud.

5. Implementation Roadmap

1. **Architecture Setup:** Initialize Firebase with security rules that restrict data access based on user roles (Driver/Passenger/Admin).
2. **Authentication & Compliance:** Implement phone-based OTP login and the Document Upload portal.
3. **Core Ride Engine:** Build the request matching flow using Geohash to ensure efficient driver discovery.
4. **UI/UX Layer:** Implement the "Professional and Engaging" interface.
5. **Safety & Utilities:** Integrate the SOS, live-sharing web links, and demand heatmaps.
6. **Load Testing:** Simulate high-concurrency environments to ensure the system handles concurrent ride requests without latency.

*The success of this project depends on the efficiency of your **Real-time Engine**. Do not rely on standard Firestore queries for finding nearby drivers; implement geospatial indexing to maintain the "fast and direct" requirement.*

This correction is critical for your financial model. By treating the **1–4 DZD surcharge** as a pure platform fee, you ensure the company captures a consistent overhead recovery fee before the driver's commission split is even calculated.

Here is the updated **Financial & Performance Logic** section for your project specification.

6. Financial & Performance Logic (Revised)

A. Dynamic Commission Model

The platform operates on a performance-based tiered commission structure to incentivize high-quality service and long-term driver retention.

- **Tier 1 (Base):** Company takes 15% commission; Driver receives 85% of the fare.
- **Tier 2 (Elite):** Company takes 7% commission; Driver receives 93% of the fare.
 - **Threshold Criteria:** A minimum of **100 completed rides** AND an average rating of **4 stars or higher**.
- **Implementation:** The system must automatically update the driver's commission rate in the database once the `rides_completed` and `average_rating` fields satisfy the criteria.

B. Fare Calculation & Company Revenue Flow

The pricing engine must strictly adhere to the following order of operations to ensure the company correctly captures the surcharge and its commission.

1. **Base Fare Calculation:** Calculated based on distance and time.
2. **Platform Surcharge:** A fixed amount of **1 to 4 DZD** is added to the total. This amount is **100% captured by the company** and is not part of the driver's share.
3. **Final Fare:** `(Base Fare) + (1 to 4 DZD Surcharge)`.
4. **Commission Deduction:**
 - The company deducts its commission percentage (10% or 20%) from the **Base Fare**.
 - The company keeps the **full 1–4 DZD surcharge**.
5. **Driver Payout:** `(Base Fare - Commission)`.

C. Driver Wallet & Pre-paid Commission

To ensure the company collects its fees (especially for cash-based rides), the platform uses a **Driver Wallet** system.

- **Wallet Requirement:** Drivers must maintain a positive balance.
- **Commission Deduction:** Upon ride completion, the company's commission (10% or 20% of the Base Fare) plus the surcharge (1–4 DZD) is deducted from the driver's virtual wallet balance.
- **Top-up System:** If the wallet balance falls below a threshold, the driver is restricted from receiving new ride requests until they perform a top-up via local Algerian payment methods.

Technical Implementation Notes for Students:

Important: To ensure the 1–4 DZD surcharge is handled correctly, the backend must generate a unique `transaction_id` for every ride that clearly separates `Fare_Revenue` (shared) from `Surcharge_Revenue` (company-only).

- **Example Calculation (Assuming Tier 1 - 15% Commission):**
 - **Base Fare:** 500 DZD
 - **Surcharge:** 3 DZD
 - **Total Passenger Pays:** 503 DZD
 - **Company Takes:** $(15\% \text{ of } 500) + 3 = 78 \text{ DZD}$

- **Driver Receives:** 422 DZD
- **Database Schema Update:**
 - **drivers** collection: Must include **wallet_balance** (Double), **rides_completed** (Integer), and **rating_average** (Double).
 - **transactions** collection: Ensure the logs show the breakdown: **[Base Fare]**, **[Surcharge]**, **[Commission Deducted]**, and **[Net Payout]**.

For a ride-hailing app like **MICHWAR**, you should avoid thinking in terms of a "number of pages." Instead, think in terms of **Core Workflows**.

Because your app serves two distinct user roles (Passenger and Driver) within one application, the navigation should be **role-based**. When a user logs in, the app determines their role and loads the corresponding "Home" environment.

Here is the essential page structure for a production-ready MVP:

1. Authentication & Onboarding (Shared)

- **Splash/Welcome Screen:** Branding and entry point.
- **Phone/OTP Verification:** Secure login (Essential for trust).
- **Role Selection/Detection:** Automatically routes the user to the appropriate mode.
- **Document Upload (Driver Only):** Dedicated portal for submitting license, *Carte Grise*, etc., for backend admin verification.

2. Passenger Mode (The User Workflow)

- **Main Map Home:** Interactive map showing current location and available nearby drivers.
- **Ride Request Flow:**
 - **Destination/Pickup Input:** Search bar + suggested locations.
 - **Ride Options:** Select vehicle tier (e.g., Standard, Premium).
 - **Booking Confirmation:** Final view of fare estimate and "Request" button.
- **Active Ride Screen:** Live map view with driver progress, trip duration/ETA, and the **SOS button** / **Live Share link** integration.
- **Post-Ride/Receipt Screen:** Rating system and breakdown of costs.
- **Profile/Loyalty Dashboard:** View earned "MICHWAR Points" and progress toward Tier 2 status.

3. Driver Mode (The Provider Workflow)

- **Main Map Home (Status Toggle):** Toggle button (Online/Offline) and Heatmap overlay.
- **Ride Request (Incoming):** Modal or full-screen overlay for incoming pings, showing destination, distance, and fare.
- **Navigation Screen:** Real-time routing to the passenger's pickup and drop-off points.
- **Earnings Dashboard:** A detailed report of daily/weekly income, tips, and current wallet balance.
- **"Heading Home" Settings:** A specialized screen or toggle in the settings drawer for directional ride filtering.

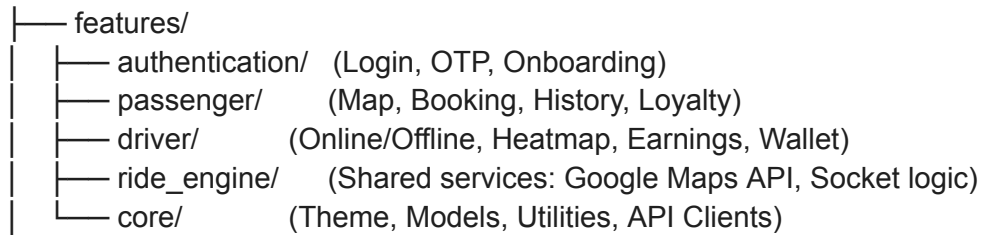
4. Settings & Shared Utilities

- **Global Settings:** Language, Notification preferences, and SOS contact list management.
- **Support/Help Center:** FAQs and direct links to admin/support channels.
- **Wallet/Transaction History:** Where drivers manage their "Top-up" balance and see commission deductions.

Pro-Tip for your Flutter Architecture:

To keep your app "stunning and fast," use **Feature-First folder structure** in Flutter rather than a massive "pages" folder. Group your code by business logic:

lib/



Why this structure works for MICHWAR:

1. **Scalability:** If you decide to add a new feature (like a "Package Delivery" mode), you just add one new directory under **features/** without touching the existing ride logic.
2. **Performance:** Since the Driver and Passenger modes are separate features, you can use **Lazy Loading** to only load the components the current user needs, keeping the app snappy.
3. **Maintenance:** The **ride_engine** acts as your "Single Source of Truth." If the backend pricing logic changes, you only update it in one place, and it propagates to both passenger and driver views instantly.